Presentation flow and script

A 45-minute walkthrough for engineers joining the project: what the system does, who may do what, how a request moves, how the code is arranged, and how mail leaves the building.

Read this alongside the five reference documents; every segment below names the one it draws from, so a question you cannot answer in the room has a page to point at. The script is written to be spoken, not read out — take the sentences as the argument to make, not as lines to recite.

> **One rule for the whole session: no live coding and no repository tour.** The point is the five decisions a newcomer cannot infer from reading the code. Setup and the runbook are homework — local setup and operations stand on their own.

1 The shape of the session

Time	Segment	The one thing they should leave with
0:00	Framing · 3 min	Where the truth lives: the code, then these documents
0:03	What the system does · 6 min	One reservation row is one trip; site is a notification filter
0:09	Roles and the whitelist · 9 min	Two roles, chosen by door; capability is re-read, never trusted
0:18	Reservation lifecycle · 9 min	Five statuses, server-side transitions, CHECK constraints behind them
0:27	Architecture and stack · 12 min	Pure rules, repositories, services — and where to put new code
0:39	Email pipeline · 8 min	Mail is enqueued, not sent; the stub succeeds while delivering nothing
0:47	Close and first task · 3 min	What to read tonight, what to run tomorrow

Fifty minutes with questions taken as they come. If the room is quiet, the architecture segment is where to spend the slack; if it is talkative, cut the migrations table and the token-contract detail first — both are documented and neither is load-bearing for a first week.

2 Framing · 3 min

Show: nothing. Say it.

> "This is an internal app for booking company vans. An associate requests a trip, an admin approves it and assigns a driver and a van, and everyone involved gets email. That is the whole product. It is small, and almost all of the difficulty is in rules you cannot see from the file tree."
>
> "So today I am not going to walk the repository. I am going to give you the five things you would otherwise learn by getting them wrong. There are five documents behind this session and every one of them cites the code that enforces what it claims, because a rule and its enforcement drift apart the moment either is edited alone."
>
> "One convention worth stating up front: where a document and the code disagree, the code is right and the document is a bug. If you find one, fix the document in the same pull request."

Do not open with the stack list. It invites twenty minutes of tooling opinions before anyone knows what the software is for.

3 What the system does · 6 min

Source: domain §1, §7. **Show:** the booking wizard and the master list, signed in as an admin.

> "Two sites operate today, Iloilo and Manila. Site is on almost every row you will read, and it is a notification filter, never a permission boundary. A Manila admin can approve an Iloilo trip. That is deliberate, and it is the assumption people get wrong first."
>
> "One row in `reservations` is one trip. There is no parent booking table. The wizard can submit up to ten trips at once and each one gets its own reference number and its own life from then on."
>
> "A trip has two ride modes. A pickup is point to point. A standby holds a van and a driver for a block of time, and it needs an end time and an approving tower head. The database enforces the shape of each one, so you cannot half-fill a standby."

> "Drivers and vans are separate rosters, because a van is assigned per trip and not owned by a driver. Either side of an assignment can also be a rental, which is free text typed in at approval time — there is no rentals table. That one decision is why driver workload and utilisation can only count roster-sourced assignments: a rental has no identity to group by."

Expect to be asked:

- "Why is a rental not a table?" — Because nothing joins on it and the client revises it per trip. Say the trade-off out loud: reporting pays for it.
- "Can two trips take the same van?" — Yes. Nothing checks for overlapping assignments. It is a known product gap, in operations §7.

4 Roles and the whitelist · 9 min

Source: domain §2–§3, architecture §8. **Show:** sign in twice with the same Domain ID — `AM65108` at `/login`, then at `/admin/login`.

> "There are exactly two roles: associate and admin_support. One flat admin role, no per-site scoping. And there is no eligibility allowlist for signing in — the directory authenticates only active employees, so a successful identify is the admission check."
>
> "What the whitelist decides is capability, not admission: whether an authenticated employee can act as an admin, and which admins get copied on mail for a site."
>
> "Now watch this. Same person, same password, two doors. The requestor door always gives you associate, even if you are whitelisted. The admin door requires admin_support and 403s otherwise. So one human can hold both roles and picks by URL."
>
> "On top of the role sits one boolean, super_admin. It decides who may edit the whitelist itself. Two people hold it today, by seed — and that restriction is a data fact, not a code rule. Any holder can grant it to a third person."
>
> "Here is the part I want you to remember. The session cookie carries a super_admin flag and nothing in the app authorizes on it. It is cosmetic. Every consumer re-reads the database, because a capability baked into a signed cookie lags in both directions: it outlives a revocation, and it misses a grant. We shipped that bug during development — a freshly promoted admin saw the tab and got a 403 on every write behind it for the life of the cookie."

> "Four guardrails protect the whitelist, and they live inside the repository layer, inside the transaction, so no future caller can route around them: no demoting yourself, no deactivating yourself, never taking the count of active super_admins to zero, and never leaving a row with no email and no Domain ID."

If someone asks how the last-holder lock works, this is the one place worth going deep — it is the subtlest code in the project:

> "It takes an advisory transaction lock before any row lock. A row lock alone is not enough: two transactions demoting different holders never contend on a row, so under READ COMMITTED both read 'two holders left' from their own snapshot and both proceed, leaving zero. Serialising every write that can shrink the count is what makes the check answer truthfully at commit time."

Expect to be asked:

- "How fast is revocation?" — Up to eight hours. The role rides in the session JWT and there is no session table. Say it plainly; it is a known hazard, not a secret.
- "What about offboarding?" — Same answer, plus operations §7. The super_admin flag is exempt because it is re-read per request; the base role is not.

5 Reservation lifecycle · 9 min

Source: domain §4–§5, §8. **Show:** approve one seeded pending trip in the trip drawer, then reassign its driver, then open the audit log.

> "Five statuses: pending, approved, approved-reassigned, rejected, cancelled. Rejected and cancelled are terminal — no path leads out of them."
>
> "Approved-Driver Reassigned is admin bookkeeping only. Every requestor-facing surface collapses it back to Approved; a requestor never sees the word Reassigned. They learn about the change from a driver-assignment email instead. Watch — I approve, then I move the driver, and the admin list changes while the requestor view does not."
>
> "Approval requires both a driver and a van, roster or rental. That rule is in the write path and mirrored by CHECK constraints, so it holds even against a direct SQL write. Rejection requires a non-blank reason."

> "Two subtleties. First, reassignment is sensitive to identity, not to edits: the status only flips if the driver or the van actually moved, so correcting a rental driver's mobile number leaves it alone. Second, a requestor cancelling someone else's request gets a 404, not a 403 — deliberately indistinguishable from a missing record, so nobody can enumerate reference numbers."
>
> "Every write uses optimistic concurrency on a version column. A zero-row update surfaces as VERSION_CONFLICT rather than a silent overwrite."
>
> "Now the audit log. There is no audit_log table. What you are looking at is a read composition over the event trails the write paths already keep — reservation events filtered to admin actions, plus roster events. A second log would be a second thing to keep in sync."
>
> "The rule that must never regress: a row change and its event are written in one transaction, always. For roster events the type system enforces it, because the recording function takes a Transaction and refuses a plain database handle. An audit trail with gaps is worse than none, because it is trusted."

If TAT comes up — and with anyone who has seen the dashboard, it will:

> "TAT is submission to the first driver assignment. First one counts, even if the driver is later reassigned; a reassignment does not restart the clock. SLA is twelve hours. And both definitions are ours, not the client's — neither term appears in the specification, and the question was deferred to stakeholders who never answered. If the client defines TAT differently, that is a code change and it is cheap. Do not treat those numbers as requirements."

6 Architecture and stack · 12 min

Source: architecture §2–§7, §9–§10. **Show:** the layering diagram, then one pure-rules file and one repository side by side.

> "Next.js App Router over Postgres, with no ORM. Kysely as a typed query builder, raw SQL where it reads better, and the schema types are generated from the live database — never hand-edited. Tailwind v4 with no JS config, shadcn on Base UI, TanStack Query for refetch and mutation, Biome instead of ESLint and Prettier."

> "Three kinds of file inside a module, and this is the map you actually need. Pure rules: no I/O, no React, no framework imports — that is where the domain lives and it is trivially testable. Repositories: they take a Kysely instance or a transaction and own the SQL. Services: they orchestrate the two. When you add code, decide which of those three you are writing before you type anything."
>
> "Layering runs one way — app to modules, modules to db and lib, nothing reaches upward. Be honest about this: no lint rule enforces it. It holds by inspection, and lib has zero framework imports today. There is exactly one intentional exception in modules, the session reader, and it needs request headers."
>
> "Errors are a Result type, converted at the route boundary, with one map from error code to HTTP status. Module error types are extracted from that map rather than declared separately, so deleting a code breaks compilation at every consumer instead of letting the two drift."
>
> "Two things on the frontend that surprise people. Every page is a server component — all thirteen of them — with client leaves for forms, drawers and charts. And every page repeats its own session check, on purpose: a layout does not re-execute on client-side navigation between sibling routes, so a layout-only check would be a fail-open. If you add an admin page, copy the block."
>
> "Mutations are never retried. Replaying an approve or a cancel is worse than surfacing the failure."

The one gotcha worth twelve minutes of anyone's day. If the room is technical, this is the highest-value ninety seconds in the session:

> "A declared design token is not a used token, and grep will lie to you three different ways: Tailwind emits the custom property whether or not anything references it, a hand-written colour mix gets baked into a flat literal at build time, and our class merger silently drops custom font-size classes because it reads them as colours. So verify against the compiled selector and the rendered HTML, not the declaration."

Skip unless asked: the migrations table, the wire-schema strictness detail, and the edge-runtime reason the middleware file must not be renamed. All three are written down; none is needed to make a first change safely.

Close the segment on the conventions table in architecture §10: "Most of these exist because their absence caused a real bug. The second column tells you what actually stops a regression, and it says 'nothing' where the answer is nothing. Read that column before you refactor."

7 Email pipeline · 8 min

Source: email §1, §8–§11, domain §9. **Show:** the outbox table after the approval you did earlier, and the dev console line from the stub.

> "One rule decides this whole design: mail is enqueued during a request, never sent. The write path appends to an outbox inside the same transaction as the row change, and a dispatcher drains it afterwards. A requestor never waits on an email provider, and a provider outage never fails a booking."
>
> "Two things drain the queue and they run the same function. Every write route dispatches after the response. And a scheduled sweeper endpoint does the same thing on a timer — that one is the only thing that fires when nobody is writing, which is exactly what you need for mail queued while the provider was down."
>
> "Recipients come from one function, so a new template cannot invent its own routing. Active whitelist rows with notify set, whose site matches the trip or is 'all', sorted by name so the Cc header is stable, and the acting person is always excluded — nobody is copied on a notice about their own action."
>
> "Retry: transient failures back off, doubling to an hour, capped at eight attempts. Permanent failures are marked failed immediately. An unknown thrown error is assumed transient, because discarding a notification over a bug in error handling is the worse failure."
>
> "Here is the one that has actually bitten us. Temporary AWS keys need a session token. Without it the provider answers 403 on every send, and 403 used to map to a transient code — so the dispatcher retried for hours and reported a misconfiguration as an outage. It now maps to permanent, and the row is marked failed with the provider's own exception name attached."
>
> "And the quiet one: in stub mode every notification is queued, rendered, and marked sent. Nothing is delivered. An environment that never switches the mode looks healthy in every log and every table while delivering nothing at all. That is why the stub refuses to construct in production."

Expect to be asked:

- "What alerts on a failed send?" — Nothing. One error log line, and no UI surfaces it. The queries to run by hand are in operations §2.
- "Is the in-app bell wired up?" — No. The rows are written and correct; nothing reads them, and the bell toasts a placeholder.

8 Close and first task · 3 min

> "Five things to carry out of the room. Site filters notifications and gates nothing. One row is one trip. Capability is re-read from the database, never trusted from a cookie. A row change and its event are one transaction. And mail is enqueued, not sent."
>
> "Tonight, read the setup document and get the app running — it is one environment variable and six commands, and everything else already points at local Postgres and the stubs. Tomorrow, sign in as all three of the identities in there, including the one with no bookings, so you have seen the empty state. Then pick anything from the known-hazards list and tell me how you would fix it."

Hand out, in this order:

1. Local setup — tonight's homework.
2. Domain rules — read before the first ticket.
3. Architecture — read before the first pull request.
4. Email and notifications and operations — reference, when the thing they describe happens.

9 Before you present

- [ ] Run `pnpm db:seed`, then `pnpm db:seed:dev`. Without the second, there are no reservations to demonstrate; without the first, the Admins tab is invisible and the roles segment falls apart.
- [ ] Have one pending trip reserved for the live approval, and know which one it is.
- [ ] Open two browser profiles ahead of time, one signed in at each door, so the two-role demonstration takes ten seconds and not two minutes.
- [ ] Have the dev console visible for the email segment — the stub's log line is the only visible evidence that mail was rendered.

See also

- Local setup — local development
- Domain rules — the business rules
- Architecture — how the code is laid out

- Operations — environment, deploy, runbook
- Email and notifications — the notification pipeline and the templates